

Week 3 Worksheet: The Transformer, Interview-Ready

CS 5090 · Software Engineering with Generative AI · covers Weeks 2–3 · every question below is the kind technical interviews actually ask

Name: _____ Date: _____

Part A. Quick checks (circle one each)

- In self-attention, the softmax of the query–key scores produces:
 - the next-token probability distribution over the vocabulary
 - weights saying how much each position contributes to this position's new representation
 - the gradient of the loss with respect to the embeddings
 - a normalized copy of the value vectors
- Masked (causal) self-attention exists so that:
 - attention runs faster on long sequences
 - the model ignores padding tokens
 - a position cannot attend to positions after it, matching how generation works
 - two heads never attend to the same position
- Match each architecture to its natural use (write a/b/c next to each):
encoder-only (BERT) _____ decoder-only (GPT) _____ encoder–decoder _____
(a) generating code left-to-right (b) translation: read the whole source, then write the target (c) encode an input and predict from the embedding, e.g. classification
- True / False: in multi-head attention, all heads share one set of Q/K/V projections, and W_O simply averages their outputs.
- One Transformer block, in order, is:
 - FFN $\rightarrow$ softmax $\rightarrow$ attention
 - self-attention then a feed-forward network, each wrapped in a residual connection and layer normalization
 - attention only: FFN appears only in the final layer
 - embedding $\rightarrow$ attention $\rightarrow$ unembedding, repeated N times
- During generation, the KV cache stores:
 - the model's weights, so they load once
 - the keys and values already computed for earlier tokens, so each new token attends to them without recomputing the whole prefix
 - the k most probable next tokens
 - the tokenizer's vocabulary

Part B. Work it by hand

7. Four tokens t_1 t_2 t_3 t_4 in a **decoder** (causal) block. Mark every cell where attention is **allowed** with a check:

	attends to t_1	t_2	t_3	t_4
t_1				
t_2				
t_3				
t_4				

8. One query attends over three keys with (scaled) scores **2, 1, 0**. Using $e^2 \approx 7.4$, $e^1 \approx 2.7$, $e^0 = 1$, compute the three attention weights (two decimals is fine), and say which token dominates the output.
weights: _____ dominant token: _____

Part C. Interview short answers (1–3 sentences each)

9. "Attention Is All You Need": what did the Transformer *remove* from previous sequence models, what replaced it, and why did that make training dramatically faster?

10. Attention by itself treats its input as an unordered set. What component fixes this, and give one reason the original paper preferred *fixed* (sinusoidal) encodings over *learned* position embeddings.

11. In a translation Transformer, self-attention and cross-attention appear in the decoder. What does each one attend to? Could you swap them and still translate: why or why not?

12. What does layer normalization do for training, and why do LLMs use *layer* norm rather than *batch* norm?

13. Attention lets a token gather information from other tokens. What is the feed-forward network's job, and why must it contain a non-linearity?

Part D. Hands-on: set up and run an AI coding agent (~20 min, pairs encouraged)

mini-SWE-agent is a coding agent whose entire brain is about a hundred lines of Python: a loop that asks an LLM for the next shell command, runs it, and appends the output to the conversation. Today's goal is simple: get it running on a small software engineering task, and use the run to practise the **golden rules of AI coding** from the lecture.

Setup (one laptop per pair; everything below is free). Get a free API key at **aistudio.google.com** (sign in → Get API key), then:

```
pip install mini-swe-agent
export GEMINI_API_KEY=<your key>      # Windows: set GEMINI_API_KEY=...
mkdir agent-demo && cd agent-demo
mini -m gemini/gemini-2.5-flash -l 0 \
  -t "Write date_utils.py with a function that parses dates like
      2026-09-18 and 09/18/2026, write pytest tests for it, run
      the tests, and fix anything that fails"
```

The free tier easily covers this exercise. No Google account? Fully local instead: install Ollama, then `ollama pull qwen2.5-coder:7b` and use `-m ollama/qwen2.5-coder:7b` (a small local model fumbles more: watching how is instructive). Either way, the agent asks before executing each command: **read every command before you confirm it**. That is the exercise.

14. Give the agent a small software engineering task (the one above, or your own), watch the whole run, and write a short reflection: what did the agent actually do, and what golden rules of AI coding does your run suggest?
